Reinforcement Learning Project Report

Snow plow problem with multi-agent reinforcement learning

Authors:

Martin Nilsson, *School of Engineering*, Jönköping University

Besan Ewir, *School of Engineering*, Jönköping University

Vladimir Giustacchini, *School of Engineering*, Jönköping University

Jorge Cuello, *School of Engineering*, Jönköping University

October 20, 2026

Contents

1 Project Overview: The Snowplow Problem **3**

2 Environment **3**

2.1 Rules 4

2.1.1 Traffic rules 5

2.2 Actions 5

2.3 Observations 6

2.4 Truncation Dynamics 6

2.5 Rewards 7

3 Algorithms **7**
3.1 PPO 7
3.2 MAPPO 8

4 Experiments **8**
4.1 Hyperparameters 8
4.2 Experiment 1: 1-Agent + Full Observation 9
4.3 Experiment 2: 1-Agent + Partial Observation 9
4.4 Experiment 3: 5-Agents + Full Observation 9
4.5 Experiment 4: 5-Agents + Partial Observation 10

5 Results **10**
5.1 Results of experiments 1 and 2 10
5.2 Results of experiments 3 and 4 11

6 Discussion and Future Work **12**

7 Conclusion **14**

1 Project Overview: The Snowplow Problem

We consider a snowplow problem in which a single snowplow, or a group of snowplows, is tasked with clearing snow from the road network of a city. The objective is for the snowplows to maximize the amount of snow removed from the road network while minimizing the time needed to do so. Snowplow routing is a well-studied optimization problem. For example, Kinable et al. [4] use constraint programming to plan snowplow routes based on a real road network. Unlike their approach, our agents do not follow pre-planned routes. Instead, they learn through reinforcement learning to decide where to move at each step based on what they can observe.

Our main motivation was to investigate the differences between agents with partial observability and agents with full observability, so that we could evaluate how their behavior changes depending on what they observe. We were also very interested in whether the agents could learn to cooperate even when no reward explicitly encourages cooperation. Instead, cooperation would have to be something the agents discover by themselves after thousands of steps of trial and error.

2 Environment

The city is represented as a grid as shown in 1 and 2, where each cell is either a secondary road, an obstacle, a highway, or an intersection. Each cell type has its own rules, which are explained in Section 2.1.1. Our environment builds on the multi-agent grid-world package `mosaic_multigrid` [3].

In the following, you can see the two different road network configurations that were used in this project:

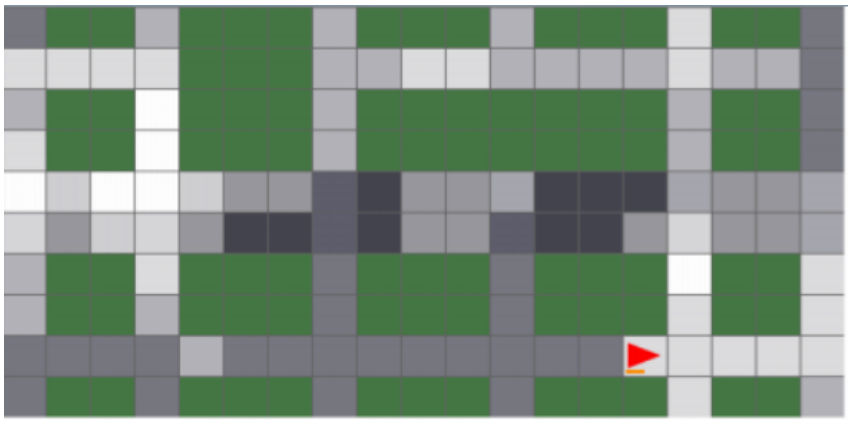


Figure 1: Road Network_1

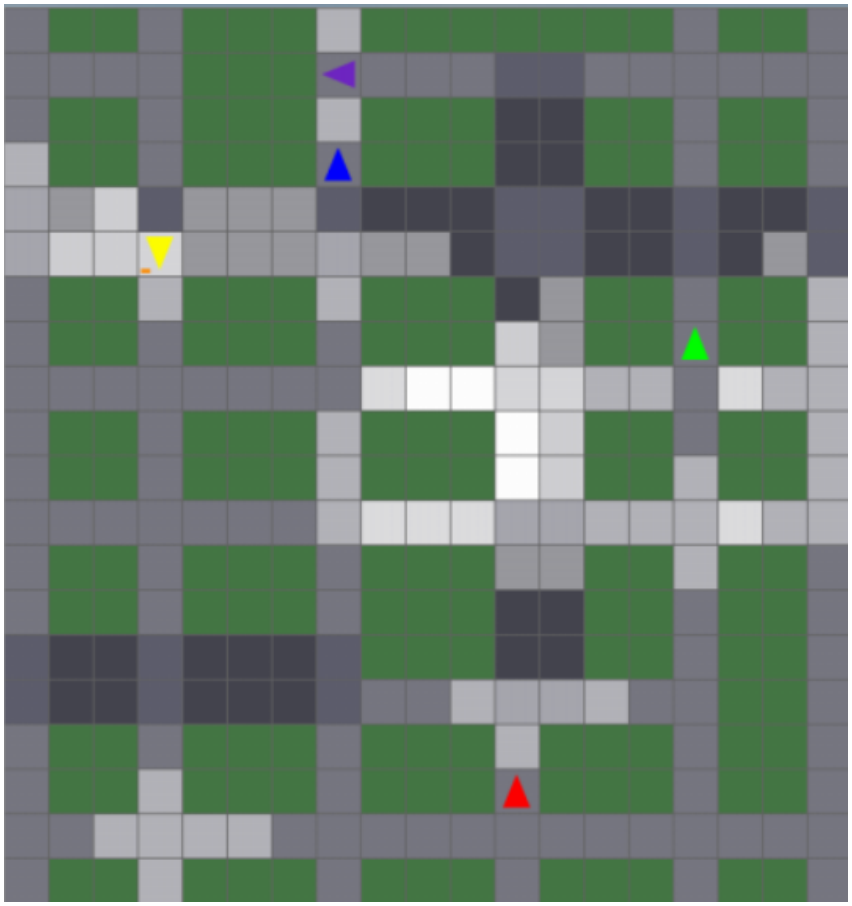


Figure 2: Road Network_3

2.1 Rules

The snowplows can move in four directions (`UP`, `DOWN`, `LEFT`, `RIGHT`) or stay in their current position (`DO_NOTHING`). The environment is dynamic: snow falls randomly onto road cells, with a cloud appearing at each step with probability

$$p_{\text{cloud}} = 0.02 \times N_{\text{agents}},$$

where N_{agents} is the number of snowplows. Scaling the probability with the number of agents balances how fast snow accumulates, so the difficulty stays fair across different map sizes and numbers of agents. The agents must adapt their strategies accordingly.

To make cooperative behaviour possible, we added rules that allow several agents to work on the same snow cell. When agents share a snowy cell, they need less time to clear it than a single agent working alone, and can therefore clear snow and earn rewards faster (see Table 1).

	Level 1	**Level 2**	**Level 3**
One snowplow	1	5	10
Two or more snowplows	1	2	3

Table 1: Number of steps needed to clear a cell, by snow level.

2.1.1 Traffic rules

The agents cannot navigate the road network however they want. There are traffic rules applied for each type of grid cell that they need to follow. The environment is initialized with the intended traffic rules for each grid cell. Different rules will apply depending on the cell type and remove invalid actions that would make the agents disobey traffic rules, go into obstacle cells, or attempt to go outside of the grid area for different traffic situations.

The types are as follows:

Cell type	**Color**	**Traffic rules**
ROAD	Grey	Any direction is allowed.
HIGHWAY	Light Grey	One-way direction only.
INTERSECTION	Dark Grey	Remove the action that normally would lead back to going in the opposite highway direction.
OBSTACLE	Dark green	Blocks the movement into that cell

Table 2: Road Network Grid Cell Types

2.2 Actions

1. $\rightarrow$: Move Right
2. $\leftarrow$: Move Left
3. $\uparrow$: Move Up
4. $\downarrow$: Move Down
5. Do nothing: Stay in the current position.

2.3 Observations

We consider two types of observation:

- **Local observation:** the agent sees a 5×5 window centred on itself. It knows the snow level of the cells around it, its own position on the map, and, if another snowplow is within its field of view, the position of that snowplow.
- **Full observation:** the agent sees the whole map, including the positions of all snowplows, the amount of snow on each road segment, and any obstacles or traffic conditions that may affect the movement of the snowplows.

2.4 Truncation Dynamics

An episode ends if a maximum number of time steps is reached. The truncation dynamics ensure that the agents learn to optimize their actions within a limited time frame, promoting efficient snow clearing strategies. In our case, we set the maximum steps per episode to 1000.

2.5 Rewards

We wanted to use as few rewards as possible, in order to study how the agents “find things out” by themselves, without forcing them to behave cooperatively. As outside observers, we know that the snowplows would clear the streets much faster if they cooperated, but this is something the agents have to realize on their own, after thousands of attempts.

The reward therefore comes only from clearing snow. An agent receives a reward of $+1$ for each snow layer it removes, so clearing a cell with the maximum snow level (snow_level $= 3$) gives a reward of $+3$.

3 Algorithms

3.1 PPO

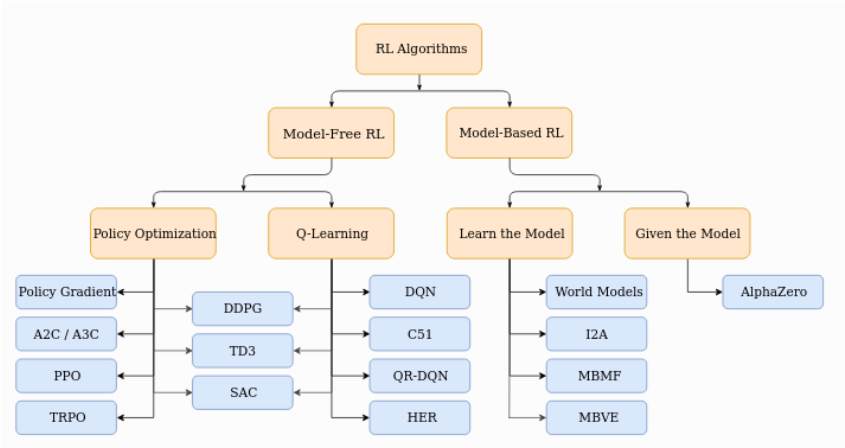


Figure 3: RL Algorithm tree

We use the **P**roximal **P**olicy **O**ptimization (PPO) algorithm[1], a very popular reinforcement learning algorithm that uses policy gradient to learn optimal policies and has a built-in clipping mechanism that regulates how much the action-selection strategy can change over time. This generally results in the algorithm being stable during training.

3.2 MAPPO

For experiments using multiple snowplow agents, we use a Multi-Agent version of PPO (MAPPO) [2], designed for cooperative multi-agent objectives. In our case, the agents plow snow individually or collaboratively to gain the highest collective reward.

4 Experiments

4.1 Hyperparameters

Table 3 lists the hyperparameters used in all experiments. Any parameter not listed keeps its default value from the code.

Table 3: Hyperparameters shared by all experiments.

Group	Parameter	Value
Environment	Episode length (steps)	1000
	Local observation window	5×5
	Shared cells allowed	only with 2+ snowplows
PPO	Learning rate	3×10^{-4}
	Parallel environments	8
	Rollout length (steps)	64
	Hidden layer size	128
	Recurrent policy (GRU)	only with local observation
	Discount factor γ	0.99
	GAE λ	0.95
	Clipping ϵ	0.2
Training	Total environment steps	500 000
	Random seed	0

4.2 Experiment 1: 1-Agent + Full Observation

Parameter	Value
Number of plows	1
Observation	FULL
Algorithm	PPO
Recurrent	False
Road network	network_1

Table 4: Experiment 1 parameters

4.3 Experiment 2: 1-Agent + Partial Observation

Parameter	Value
Number of plows	1
Observation	PARTIAL
Algorithm	PPO
Recurrent	True
Road network	network_1
PO-range	(5×5)

Table 5: Experiment 2 parameters

4.4 Experiment 3: 5-Agents + Full Observation

Parameter	Value
Number of plows	5
Observation	FULL
Algorithm	MAPPO
Recurrent	False
Road network	network_3

Table 6: Experiment 3 parameters

4.5 Experiment 4: 5-Agents + Partial Observation

Parameter	Value
Number of plows	5
Observation	PARTIAL
Algorithm	MAPPO
Recurrent	True
Road network	network_3
PO-range	(5×5)

Table 7: Experiment 4 parameters

5 Results

5.1 Results of experiments 1 and 2

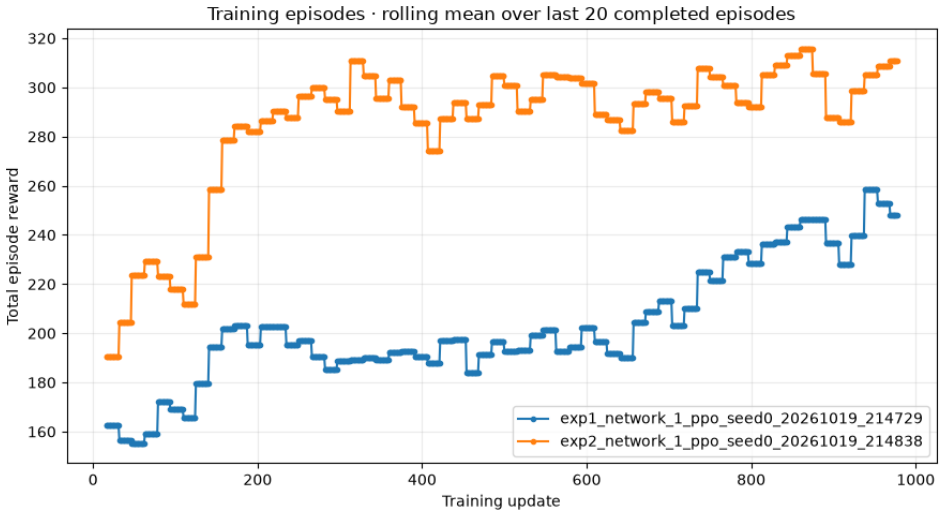


Figure 4: Reward comparison between experiment 1 and experiment 2

Metrics	Results
total_reward	239.92
steps	1000.0
fraction_clean	0.6121495327102804
snow_cleared	239.92
snow_fallen	312.08
snow_remaining	72.16
repeated_work	455.28
cells_cleared	151.04
plowing_steps	534.7
std_total_reward	38.02727442244579
episodes	50

Table 8: Experiment 1: FO-Agent metrics

Metrics	Results
total_reward	297.96
steps	1000.0
fraction_clean	0.8020560747663551
snow_cleared	297.96
snow_fallen	325.46
snow_remaining	27.5
repeated_work	372.76
cells_cleared	197.62
plowing_steps	625.74
std_total_reward	43.720457454148395
episodes	50

Table 9: Experiment 2: PO-Agent metrics

The evaluation metrics seen in 8 and 9 are averages across all 50 episodes.

In the first two experiments, we got the general results showing that the Partial Observability (PO) experiment with a single agent was able to get a higher reward compared to the Full Observability (FO) experiment.

When comparing a playthrough for each of the saved models, we noticed that the FO-agent learned a policy that would, in some scenarios, chase after some lower levels of snow instead of going for `snow_level = 3` because it was actually able to get quicker rewards for plowing `snow_level = 1` when in close proximity to them, because they take much fewer time steps to clear compared to level 3.

Meanwhile, the second experiment showed the PO-agent converging to a policy where it mostly chose not to take risks of avoiding cells that were guaranteed to give high rewards because of its limitations in the observation. Compared to the first experiment, the FO-agent could sometimes choose to turn around and go for smaller and quicker rewards instead of wasting time on the higher snow levels.

5.2 Results of experiments 3 and 4

Figure 5 shows the total episode reward during training for both experiments, averaged over the last 20 completed episodes. Several interesting observations can

be made.

First, Experiment 4 performs better than Experiment 3. This is surprising, because the agents in Experiment 4 only have partial observability. Common sense would suggest that agents with more information should perform better, but the opposite happens here. We believe this may be because, with full observability, the agents receive too much information and cannot focus on the cells that matter. In a way, much of what they observe acts as noise.

Second, the two curves follow a very similar pattern: when one goes up, the other usually goes up as well. This is most likely because both experiments were run on the same road network (`network_3`) with the same random seed, so the snow falls in the same places in both experiments.

Finally, regarding cooperation, we did not find any evidence that the agents learned to collaborate with each other. In future work, the agents should be trained for longer to check whether this behaviour eventually appears. Alternatively, adding an explicit reward for collaboration would probably encourage it, but only if the goal is specifically to make the agents cooperate, rather than to see whether cooperation emerges on its own.

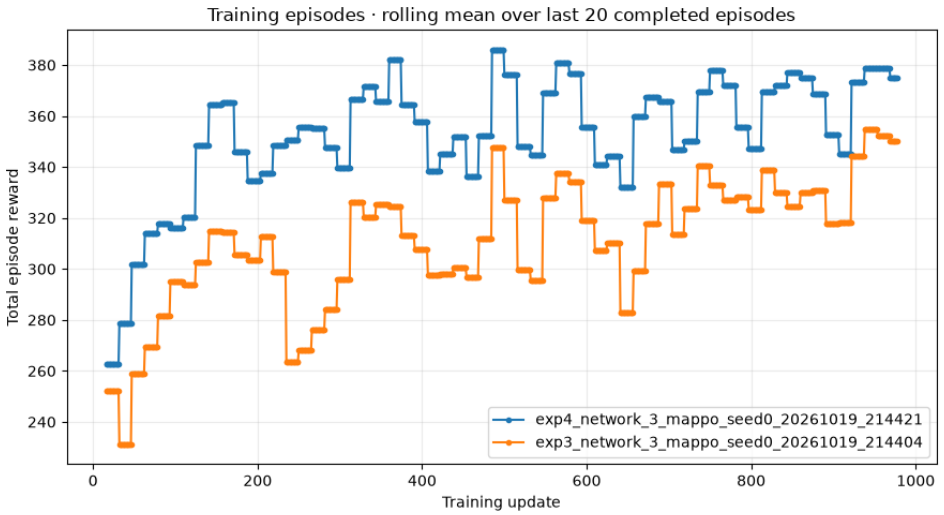


Figure 5: Reward comparison between experiment 3 and experiment 4

6 Discussion and Future Work

Our hopes for introducing `snow_levels` were that multi-agents would converge on a policy that tells them to collaborate on areas with more snow. But on the other hand,

they should also realize that it will not be optimal for all situations because working on `snow_level = 1` will still only give them +1 reward. Therefore, if they only observe level 1 cells, they would be encouraged to spread out more and cover more surface by themselves, which would earn a higher total reward than doing it together all the time. This would simulate a work-by-demand scenario that also commonly occurs in a realistic setting.

One potential shortcoming with this environment design might be that the decision-making strategy is not strongly motivated enough by the magnitude of the time steps that it takes to clear snow in relation to the reward they are getting. For example, we ask ourselves: will one agent ”give up” chasing snow cells and turn around to help another agent that ”in theory” would yield a higher reward? And unfortunately, we were not able to see this intended behavior. These dynamics might not be sufficiently expressed but could potentially be improved by adding more snow levels with linear or exponential time steps needed for clearing snow. However, this would probably also increase training time.

For future work, these are some of the main considerations that we thought of:

- **Hyperparameter tuning.** Exploring a broader range of hyperparameters, such as the learning rate, the rollout length, and the training time. The values we used are commonly used for PPO and gave stable training in our experiments. However, they were not systematically tuned. Reinforcement learning methods are known to be sensitive to their hyperparameters, so a more careful search could improve the results. In particular, a different learning rate, rollout length, or a longer training time might change how quickly, and whether, the agents learn to cooperate. We therefore leave a systematic hyperparameter search as future work.
- **Cloud spawning.** Implementing a different way of spawning clouds, where the spawn probability depends on the number of grid cells instead of the number of agents. This could give a more balanced amount of snow across maps of different sizes.
- **Work-by-demand scenario.** Analyzing a scenario in which the agents distribute their work according to demand. With the current cell-cleaning configuration, the optimal behavior would be for two agents to clean each cell together. In the single-agent experiments, the agent should instead focus on removing one layer from several cells, rather than removing several layers from a single cell.
- **A metric for work by demand.** This scenario could be measured with a metric such as the ratio between the shared team goal points and the number of steps.

We have not yet decided how the team goal points should be defined. This ratio should probably grow exponentially, be inversely proportional to the cloud probability (and therefore to the number of grid cells), and be proportional to the number of agents.

7 Conclusion

After reviewing the results, we were surprised by the fact that from experiments 1 & 2, as well as 3 & 4, the PO-agents actually were outperforming FO-agents. This points against our initial hypothesis, which was that agents that can observe the full environment would have an advantage and should be able to find a better navigation policy to get higher rewards. This could suggest that agents with full observability have a hard time identifying the strongest series of actions that would get the highest reward, which may be because of indecisiveness about what to prioritize based on their positions.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017, arXiv:1707.06347. [Online]. Available: https://arxiv.org/abs/1707.06347

[2] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, “The surprising effectiveness of PPO in cooperative multi-agent games,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[3] A. Mousa, “mosaic_multigrid: Research-grade multi-agent gridworld environments,” 2026, Python package, version 7.0.0. [Online]. Available: https://pypi.org/project/mosaic-multigrid/

[4] J. Kinable, W.-J. van Hoeve, and S. F. Smith, “Snow plow route optimization: A constraint programming approach,” *IISE Transactions*, vol. 53, no. 6, pp. 685–703, 2021.